

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

실무 중심 이론, 실무 중심 프로젝트로 단단하게 시작하자

성낙현의 JSP 자바 웹 프로그래밍



우리는
가치가 성장하는
시간을
만듭니다.



[1단계] 빠르게 익히는 JSP 기초

00 개발 환경 구축

- 01 JSP 기본
- 02 내장 객체(Implicit Object)
- 03 내장 객체의 영역(Scope)
- 04 쿠키(Cookie)
- 05 데이터베이스
- 06 세션(Session)
- 07 액션 태그(Action Tag)
- [Project] 08 모델1 방식의 회원제 게시판 만들기
- [Project] 09 게시판에 페이징 기능 넣기

[2단계] 고급 기능으로 스킬 레벨업

- 10 표현 언어(EL : Expression Language)
- 11 JSP 표준 태그 라이브러리(JSTL)
- 12 파일 업로드 및 다운로드
- 13 서블릿(Servlet)
- [Project] 14 모델2 방식(MVC 패턴)의 자료실형 게시판 만들기

[3단계] 프로젝트로 익히는 현업 스킬

- [Project] 15 웹소켓으로 채팅 프로그램 만들기
- [Project] 16 SMTP를 활용한 이메일 전송하기
- [Project] 17 네이버 검색 API를 활용한 검색 결과 출력하기
- [Project] 18 배포하기

Chapter

13

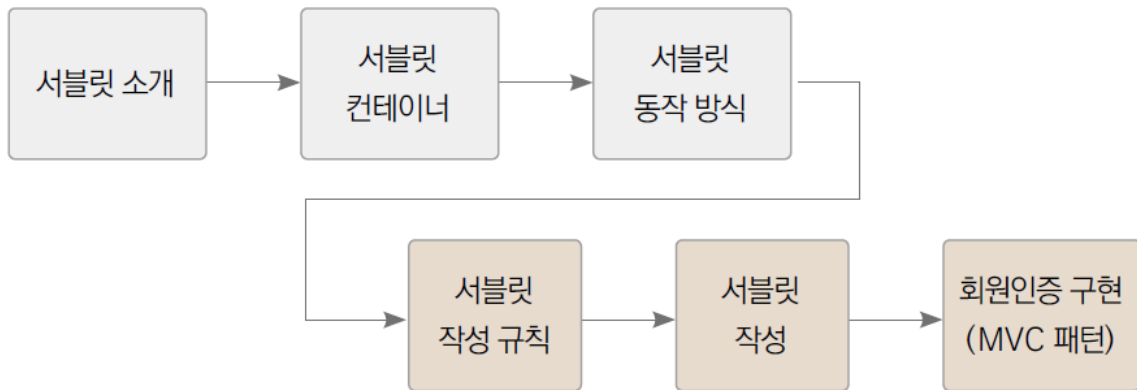
서블릿(Servlet)



□ 학습 목표

- MVC 패턴을 적용한 모델2 방식의 게시판을 제작하기 위해 필요한 기술인 서블릿을 학습
- 서블릿의 개념과 동작 방식을 이해하고, 작성 규칙에 따라 URL 요청명과 서블릿 클래스를 매핑한 후 클라이언트의 요청을 처리

□ 학습 순서



□ 활용 사례

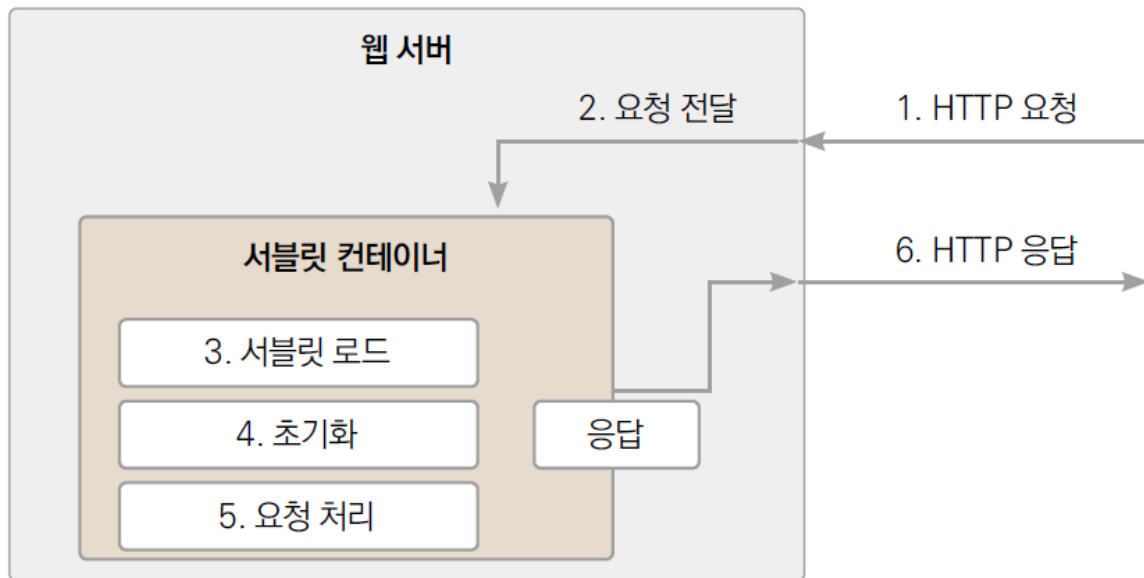
- 서블릿은 모델2 방식(MVC)으로 웹 애플리케이션을 개발하는 데 사용
- MVC 패턴을 지원하는 프레임워크에서도 서블릿과 유사한 코드가 많이 사용

13.1 서블릿이란?

- 서블릿(Servlet)은 JSP가 나오기 전, 자바로 웹 애플리케이션을 개발할 수 있도록 만든 기술
- 서블릿은 서버 단에서 클라이언트의 요청을 받아 처리한 후 응답하는 역할
- 서블릿 특징
 - 클라이언트의 요청에 대해 동적으로 작동하는 웹 애플리케이션 컴포넌트
 - MVC 모델에서 컨트롤러(Controller) 역할
 - 모든 메서드는 스레드로 동작
 - javax.servlet.http 패키지의 HttpServlet 클래스를 상속받음

13.2 서블릿 컨테이너(1)

- 서블릿과 서블릿 컨테이너

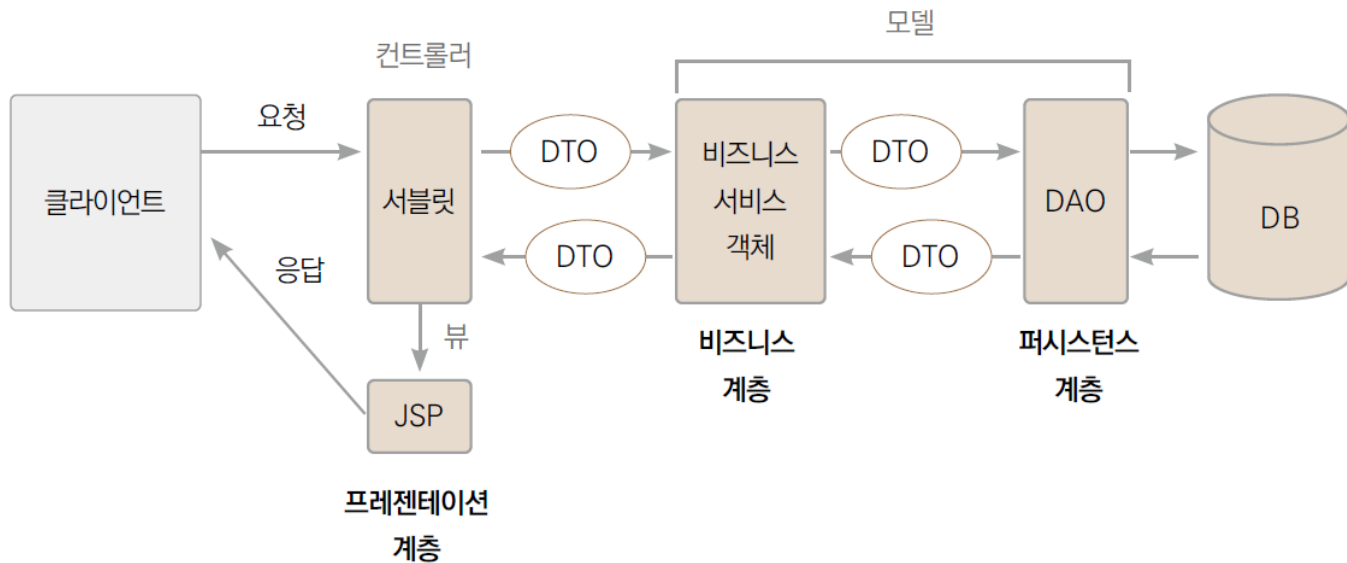


13.2 서블릿 컨테이너(2)

- 서블릿 컨테이너로 톰캣(Tomcat) 사용
 - 서블릿의 수명주기를 관리하고, 요청이 오면 스레드를 생성해 처리
 - 클라이언트의 요청을 받아서 응답을 보낼 수 있도록 통신을 지원
 - 서블릿 컨테이너 역할
 - 통신 지원 : 클라이언트와 통신하려면 서버는 특정 포트(port)로 소켓(Socket)을 열고 I/O 스트림을 생성하는 등 복잡한 과정이 필요 - 서블릿 컨테이너는 이 과정을 간단히 해주는 API를 제공
 - 수명주기 관리 : 서블릿을 인스턴스화한 후 초기화하고, 요청에 맞는 적절한 메서드를 호출하고 응답한 후에는 가비지 컬렉션을 통해 객체를 소멸
 - 멀티스레딩 관리 : 서블릿 요청들은 스레드를 생성해 처리. 즉, 멀티스레드 방식으로 여러 요청을 동시에 처리
 - 선언적인 보안 관리 및 JSP 지원 : 서블릿 컨테이너는 보안 기능을 지원하므로 별도로 구현하지 않아도 됨

13.3 서블릿의 동작 방식(1)

- 서블릿 동작 방식



13.3 서블릿의 동작 방식(2)

- 서블릿은 MVC 패턴에서 컨트롤러(Controller) 역할
 - ① 클라이언트의 요청을 받음
 - ② 분석 후 요청을 처리할 서블릿을 탐색
 - ③ 서블릿을 통해 비즈니스 서비스 로직을 호출
 - ④ 모델(Model)로부터 그 결과값을 받아옴
 - ⑤ request나 session 영역에 저장한 후 결과값을 출력할 적절한 뷰(View) 선택
 - ⑥ 최종적으로 선택된 뷰(jsp 페이지)에 결과값을 출력한 후 요청한 클라이언트에 응답
- 모델2 방식: MVC 패턴을 따라 웹 애플리케이션을 개발하는 방법
- 서비스 객체는 컨트롤러가 요청을 분석한 후 호출되어 실제 비즈니스 로직을 처리하는 역할
 - 프로그램으로 구현하면 코드가 너무 어려워짐으로 여기서는 서블릿이 컨트롤러와 서비스 객체의 역할을 동시에 할 수 있도록 구현

13.4 서블릿 작성 규칙(1)

1. 기본적으로 javax.servlet, javax.servlet.http, java.io 패키지를 импорт
2. 서블릿 클래스는 반드시 public으로 선언해야 하고, HttpServlet을 상속받아야 함
3. 사용자의 요청을 처리하기 위해 doGet() 메서드나 doPost() 메서드를 반드시 오버라이딩
4. doGet() 또는 doPost() 메서드는 ServletException과 IOException 예외를 던지도록 (throws) 선언
5. doGet() 또는 doPost() 메서드를 호출할 때의 매개변수는 HttpServletRequest와 HttpServletResponse를 사용

13.4 서블릿 작성 규칙(2)

- 서블릿 클래스의 예

```
package 패키지명;
import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class 서블릿클래스명 extends HttpServlet { // HttpServlet 상속
    @Override // doGet() 오버라이딩
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        // 메서드의 실행부
    }
}
```

기본적으로 필요한
패키지(클래스)

13.5 서블릿 작성(1)

- JSP에서는 클라이언트의 요청을 JSP가 직접 받아 처리하지만, 서블릿은 요청명을 기준으로 이를 처리할 서블릿을 선택
- 매핑(mapping): 요청명과 서블릿을 연결해주는 작업
 1. web.xml에 기술하는 방법
 2. @WebServlet 애너테이션을 사용하여 코드에 직접 명시하는 방법

13.5 서블릿 작성(2)

13.5.1 web.xml에서 매핑

- web.xml을 사용하여 요청명으로 서블릿을 선택

```
<servlet> <!-- 서블릿 등록 -->
  <servlet-name>서블릿명</servlet-name>
  <servlet-class>패키지를 포함한 서블릿 클래스명</servlet-class>
</servlet>
<servlet-mapping> <!-- 서블릿과 요청명(요청 URL) 매핑 -->
  <servlet-name>서블릿명</servlet-name>
  <url-pattern>클라이언트 요청 URL</url-pattern>
</servlet-mapping>
```

13.5 서블릿 작성(3)

- 서블릿을 등록한 후, 등록한 서블릿과 요청 URL을 매핑
 - <servlet> : 서블릿 클래스를 등록
 - <servlet-name> : 서블릿을 참조할 때 사용할 이름을 입력
 - <servlet-class> : 패키지를 포함한 서블릿 클래스명을 입력
 - <servlet-mapping> : 매핑 정보를 등록
 - <servlet-name> : <servlet>에서 사용한 <servlet-name>과 동일한 이름을 입력
 - <url-pattern> : 요청명으로 사용할 경로를 입력
 - 컨텍스트 루트를 제외한, '/'로 시작하는 경로를 사용



13.5 서블릿 작성(4)

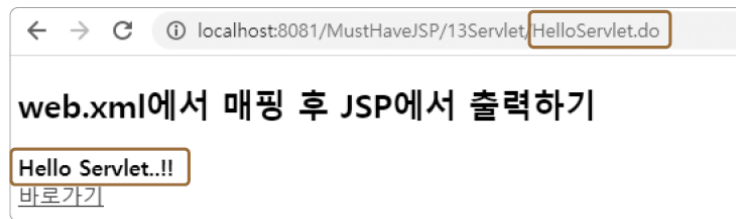
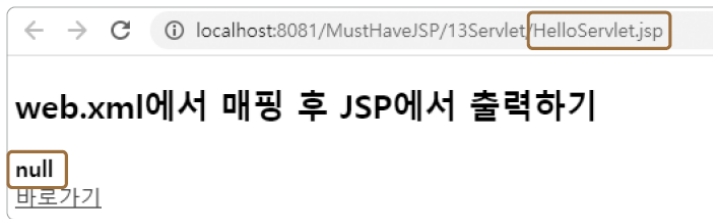
- WebContent 하위에 13Servlet 폴더를 생성한 후 예제 작성
[예제 13-1] web.xml을 이용한 매핑
 - ① request 영역에 저장된 message 속성을 출력
 - 이 속성의 값은 뒤에서 작성할 서블릿에서 저장
 - ② 바로가기 링크로, 목표 URL을 서블릿 요청명으로 지정
- web.xml에서 요청명을 서블릿으로 매핑
[예제 13-2] web.xml에 매칭 정보 추가
 - ① 서블릿을 매핑하기 위한 서블릿명을 작성
 - ② 해당 요청을 처리할 서블릿을 패키지를 포함하여 명시
 - ③ ① 과 동일한 서블릿명
 - ④ 컨텍스트 루트를 제외한 요청명을 기입
 - 요청명은 보통 .do로 끝나는 형태를 사용하지만, 다른 형태도 가능

13.5 서블릿 작성(5)

- 실제 요청을 처리할 서블릿 클래스를 작성
[예제 13-3] 요청을 처리할 서블릿 클래스
 - ① HelloServlet 클래스를 servlet 패키지에 생성한 후 HttpServlet 클래스를 상속
 - ② doGet() 메서드를 오버라이딩
 - 이클립스의 자동완성 기능을 사용하면 필요한 모든 import가 자동으로 생성
 - ③ request 영역에 message라는 속성으로 데이터를 저장
 - ④ HelloServlet.jsp로 포워드
 - req는 doGet() 메서드의 매개변수로 전달받은 request 내장 객체
 - request 영역에 저장된 데이터는 포워드된 페이지까지 공유되므로 해당 속성 출력 가능

13.5 서블릿 작성(6)

- HelloServlet.jsp를 실행
 - 서블릿은 클래스이므로 새롭게 작성하거나 수정을 했다면 반드시 서버를 재시작해야 함
 - request 영역에 저장된 데이터가 null이라고 출력
 - 단순히 JSP만 실행했으므로 request 영역에는 아직 아무런 데이터도 저장되지 않았기 때문임
- [바로가기] 링크를 클릭
 - 주소표시줄을 보면 web.xml에서 매핑한 요청명으로 변경된 것을 알 수 있고, "Hello Servlet..!!"이라는 문자열이 출력
 - 이와 같이 서블릿은 요청명을 통해 서블릿이 직접 요청을 처리한 후 → 데이터를 영역에 저장하고 → 결과를 출력할 JSP를 선택하여 → 영역을 통해 공유된 데이터를 출력하는 형식으로 클라이언트에게 응답



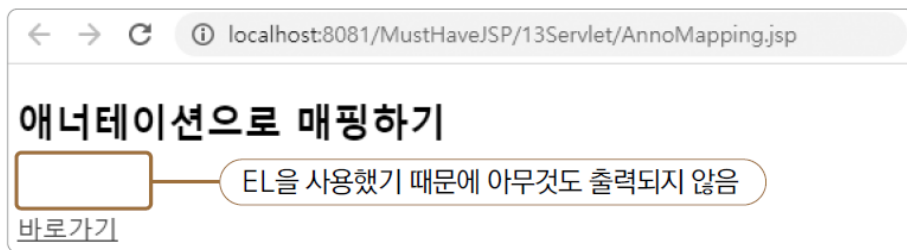
13.5 서블릿 작성(7)

13.5.2 @WebServlet 애너테이션으로 매핑

- @WebServlet 애너테이션을 사용하여 요청명으로 서블릿을 선택
- 요청명을 통한 바로가기 링크가 있는 JSP를 작성

[예제 13-4] 애너테이션을 이용한 매핑

- ① EL을 이용해 request 영역에 저장된 데이터를 출력
- ② request.getContextPath()는 컨텍스트 루트 경로를 반환
 - 프로젝트명인 "/MustHaveJSP"가 반환되며, 여기에 요청명을 합쳐 바로가기 링크 완성

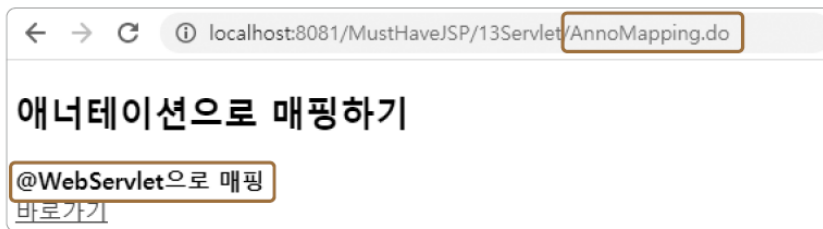


13.5 서블릿 작성(8)

- 서블릿 작성

[예제 13-5] 요청을 처리할 서블릿 클래스

- ① @WebServlet 애너테이션을 이용해 이 서블릿이 요청명 `"/13Servlet/AnnoMapping.do"`를 | 처리할 것임을 선언 - 요청명은 web.xml에서 `<url-pattern>`에 입력한 값과 똑같이, 컨텍스트 루트를 제외한 경로를 입력
- ② 이 클래스는 서블릿이므로 `HttpServlet`을 상속
- ③ `doGet( )` 메서드를 오버라이딩
- ④ `request` 영역에 데이터를 저장
- ⑤ `AnnoMapping.jsp`로 포워드



13.5 서블릿 작성(9)

- @WebServlet 애너테이션을 통한 매핑의 단점
 - 요청명(url-pattern)이 변경된다면 해당 서블릿 클래스를 수정한 후 다시 컴파일해야 함
 - 서블릿 개수가 많아지면 특정 요청명을 처리하는 클래스를 찾기가 어려움
 - web.xml을 사용하면 요청명만으로 클래스를 쉽게 찾을 수 있지만, @WebServlet 방식에서는 검색할 방법이 마땅히 없음
 - @WebServlet을 사용할 때는 요청명만으로 서블릿 클래스명을 바로 알 수 있도록 둘 사이에 명확한 이름 규칙(name rule)을 정해두는 것이 바람직

13.5 서블릿 작성(10)

13.5.3 JSP 없이 서블릿에서 바로 응답 출력

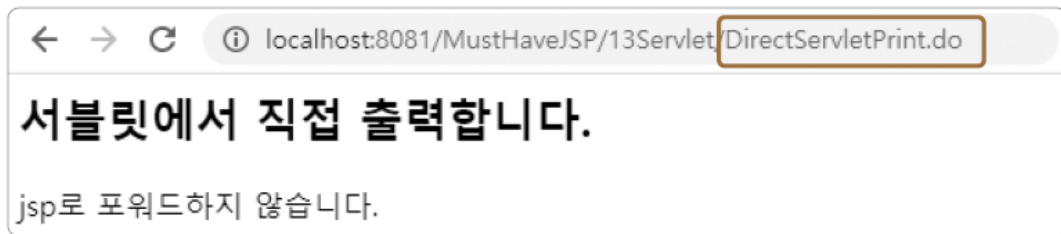
- JSP를 사용하지 않고 서블릿에서 내용을 즉시 출력
[예제 13-6] 서블릿에서 직접 응답 출력하기
 - ① post 방식으로 전송하기 위해 <form> 태그를 사용 - 요청명은 action 속성에 지정
- web.xml에 요청명과 서블릿을 매핑
[예제 13-7] 요청명과 서블릿 매핑하기
 - ① 요청을 담당할 서블릿은 servlet 패키지의 DirectServletPrint 클래스
 - ② 요청명은 "/13Servlet/DirectServletPrint.do"

13.5 서블릿 작성(11)

- 매핑한 정보에서 기술한 서블릿 작성
[예제 13-8] 응답을 직접 출력하는 서블릿
 - ① post 방식의 요청이므로 doPost() 메서드를 오버라이딩
 - ② 클라이언트에 응답하기 위해 response 내장 객체에 응답 콘텐츠 타입을 지정
 - 콘텐츠 타입은 "text/html"으로, 캐릭터셋은 "UTF-8"로 지정
 - ③ 웹 브라우저에 전송할 응답 결과를 쓰기 위해 response 내장 객체에서 PrintWriter 객체를 얻어옴
 - ④ PrintWriter는 println() 메서드를 제공 - 이 메서드를 이용해 응답 내용을 출력
 - 출력 내용은 간단한 태그들로 구성된 기본적인 HTML 문서
 - ⑤ PrintWriter 객체를 닫아줌

13.5 서블릿 작성(12)

- [예제 13-6]의 DirectServletPrint.jsp를 실행



- JSP 없이 서블릿에서 응답을 직접 출력하면 코드가 굉장히 지저분해지기 쉬움
- 대부분의 경우에는 JSP를 통해 출력하는 쪽이 편리
- 비동기 방식으로 통신할 때 XML이나 JSON을 사용하는 경우가 있으며, 이와 같이 순수 데이터만 출력해야 하는 경우에는 서블릿에서 직접 출력하는 게 편리

13.5 서블릿 작성(13)

13.5.4 한 번의 매핑으로 여러 가지 요청 처리

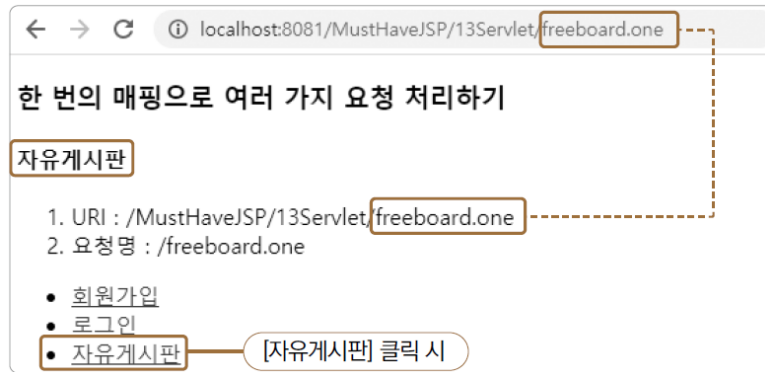
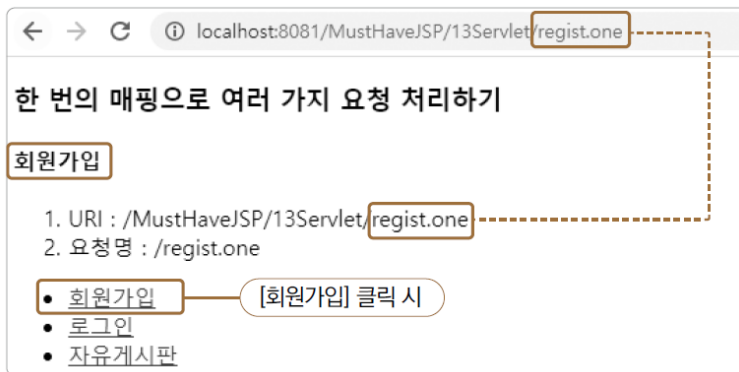
- [예제 13-9] 한 번의 매핑으로 여러 가지 요청 처리
 - ① 서블릿에서 request 영역에 저장할 결괏값
 - ② 클라이언트가 요청한 전체 경로를 표시
 - ③ 전체 경로에서 마지막의 xxx.one 부분을 추출한 문자열
 - ④ 각 페이지로의 바로가기 링크

13.5 서블릿 작성(14)

- [예제 13-10] 여러 가지 요청을 처리하는 서블릿
 - ① 와일드카드(*)를 사용하여 URL 패턴이 "*.one"에 해당하는 요청을 모두 이 서블릿과 매핑 - ".one"으로 끝나는 모든 요청명이 매핑
 - ② request 내장 객체로부터 현재 경로에서 호스트명을 제외한 나머지 부분을 알아냄
 - ③ 마지막 슬래시(/)의 인덱스를 구함
 - ④ 이 인덱스로 경로의 마지막 부분의 문자열을 얻어옴
 - ⑤ 이 문자열을 통해 페이지를 구분하여, 각 페이지를 처리할 수 있는 메서드를 호출
 - ⑥ uri와 페이지 구분을 위한 문자열(commandStr)을 request 영역에 저장
 - ⑦ FrontController.jsp로 포워드
 - ⑧ 각 페이지를 처리할 수 있는 메서드 - 각 페이지에 출력할 데이터를 request 영역에 저장

13.5 서블릿 작성(15)

- FrontController.jsp를 실행
[회원가입]과 [자유게시판] 링크 확인
 - 서블릿을 요청명 "*.one"와 매핑한 후, 마지막 슬래시로 경로명을 구분하여 담당 메서드를 호출
 - 요청명마다 매번 서블릿을 하나씩 만들지 않아도 되어서 편리함
 - 단, 서블릿 클래스의 내용이 방대해질 수 있으므로 카테고리별로 구분하여 작성하는 것이 좋음



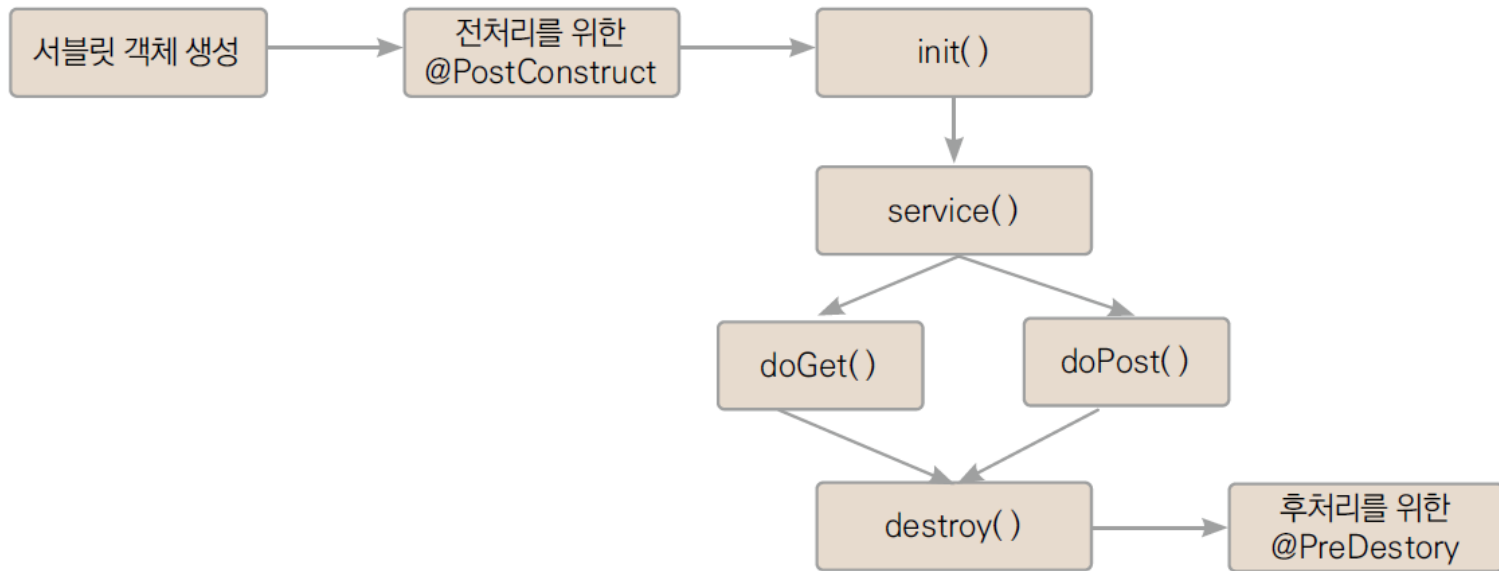
13.5 서블릿 작성(16)

13.5.5 서블릿의 수명주기 메서드

- 서블릿의 수명주기
 - 클라이언트의 요청 → 서블릿 객체를 생성 → 서블릿을 초기화 → 요청을 처리
→ 서버를 종료할 때 서블릿 객체를 소멸
 - 서블릿 컨테이너는 서블릿 객체를 생성하고 각 단계마다 자동으로 특정 메서드를 호출하여 해당 단계에 필요한 기능을 수행

13.5 서블릿 작성(17)

- 서블릿 수명주기 메서드



13.5 서블릿 작성(18)

- @PostConstruct
 - 객체 생성 직후, init() 메서드를 호출하기 전에 호출
 - 애너테이션을 사용하므로 메서드명은 개발자가 정하면 됨
- init()
 - 서블릿의 초기화 작업을 수행하기 위해 호출
 - 최초 요청 시 딱 한 번만 호출
- service()
 - 클라이언트의 요청을 처리하기 위해 호출
 - 전송 방식이 get이면 doGet() 메서드를, post면 doPost() 메서드를 호출
 - 따라서 service() 메서드는 두 가지 전송 방식 모두를 처리할 수 있음
- destroy()
 - 서블릿이 새롭게 컴파일되거나, 서버가 종료될 때 호출
- @PreDestroy
 - destroy() 메서드가 실행되고 난 후, 컨테이너가 이 서블릿 객체를 제거하는 과정에서 호출
 - @PostConstruct와 동일하게 메서드명은 개발자가 정하면 됨

13.5 서블릿 작성(19)

- [예제 13-11] 수명주기 메서드 동작 확인
 - ① 폼값을 전송해주는 자바스크립트 코드
 - 첫 번째 인수는 <form> 태그의 DOM 객체, 두 번째 인수는 전송 방식
 - ② 두 번째 인수를 보고 전송 방식을 결정
 - ③ 폼값을 전송
 - ④ <form> 태그를 정의
 - action 속성을 제외한 나머지는 클릭 시 자바스크립트에서 설정

13.5 서블릿 작성(20)

- [예제 13-12] 수명주기 메서드 동작 확인용 서블릿
 - 전송 방식을 확인해 doGet() 또는 doPost() 호출
 - [Get 방식 요청하기]를 눌렀을 때는 myPostConstruct() → init() → service() 순서로 호출
 - [Post 방식 요청하기]를 누르면 앞의 두 메서드는 호출되지 않고 곧바로 service()부터 호출



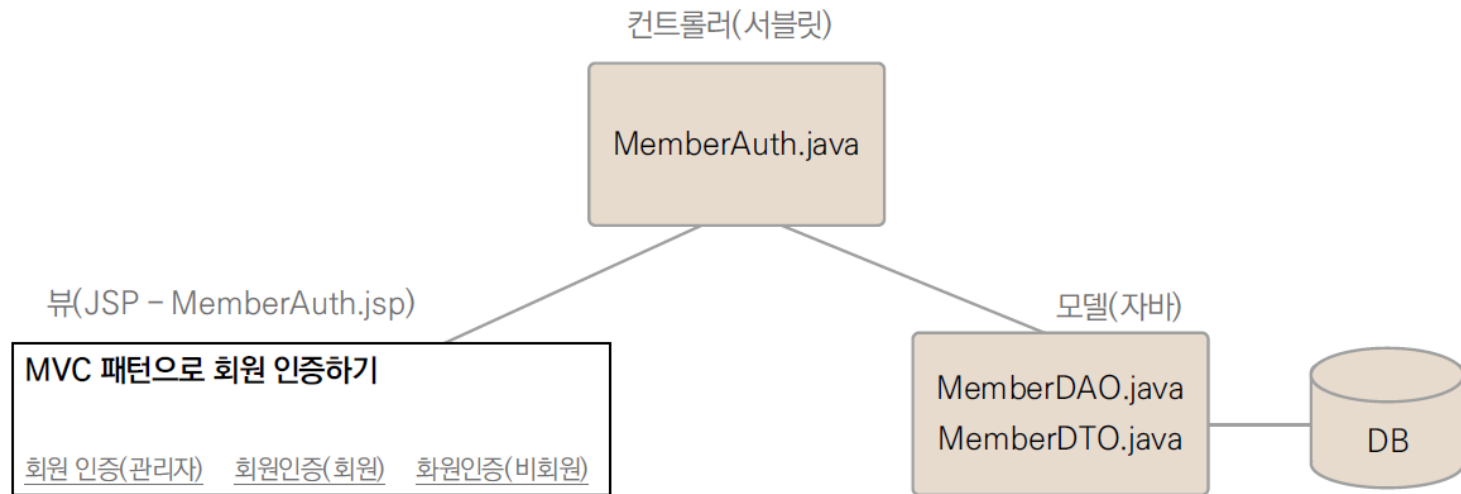
13.5 서블릿 작성(21)

- [Servers] 뷰에서 [Stop the server] 버튼을 클릭하여 톰캣을 종료하고, [Console] 뷰에서 로그를 확인
 - `destroy()` 메서드와 `myPreDestroy()` 메서드가 차례대로 실행된 것을 알 수 있음
 - 서블릿 객체는 메모리에서 완전히 소멸됨

```
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflectiv
WARNING: All illegal access operations will be denied in a future release
destroy() 호출
myPreDestroy() 호출
8월 03, 2021 7:38:45 오후 org.apache.coyote.AbstractProtocol stop
INFO: 프로토콜 핸들러 ["http-nio-8081"]을(를) 중지시킵니다.
8월 03, 2021 7:38:45 오후 org.apache.coyote.AbstractProtocol destroy
INFO: 프로토콜 핸들러 ["http-nio-8081"]을(를) 소멸시킵니다.
```

13.6 MVC 패턴을 적용한 회원인증 구현(1)

- MVC 패턴 회원인증 구성도



13.6 MVC 패턴을 적용한 회원인증 구현(2)

13.6.1 뷰(JSP)

- 아이디와 패스워드를 받아 인증 요청을 할 JSP 작성
 - 관리자, 회원, 비회원의 세 가지 아이디로 인증 요청
- [예제 13-13] MVC 패턴을 통한 회원인증용 메뉴 화면
- ① 서블릿에서 회원인증 처리 후 결과 메시지를 출력
 - ② ③ ④ 관리자 ID, 회원 ID, 비회원 ID로 인증을 요청하는 기능을 수행
- 회원 여부는 member 테이블에 존재하는 아이디인지 여부로 판단하며, 관리자 아이디는 web.xml에서 서블릿 초기화 매개변수로 설정

13.6 MVC 패턴을 적용한 회원인증 구현(3)

13.6.2 컨트롤러(서블릿)

- web.xml에서 서블릿으로 매핑
[예제 13-13]이 작성된 폴더명이 "13Servlet"이므로, 회원인증 요청명은
"/13Servlet/MemberAuth.mvc"

[예제 13-14] 서블릿 매핑

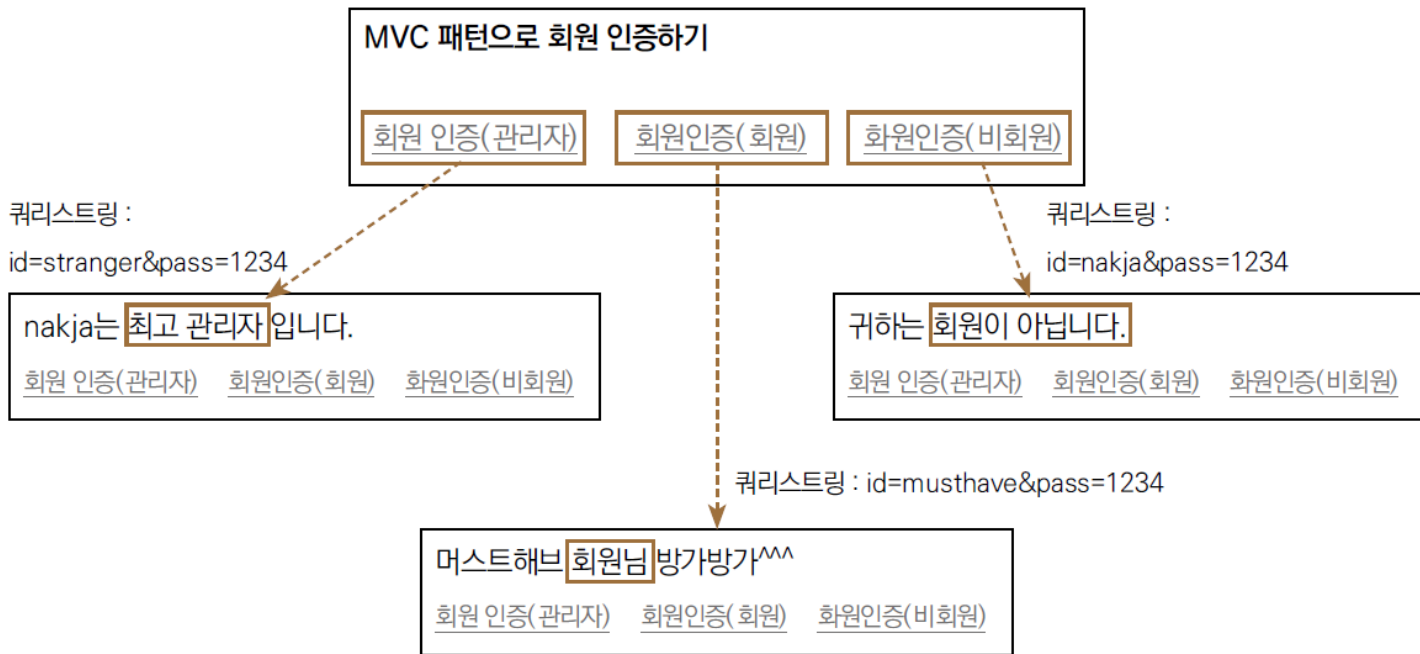
- ① 서블릿명은 "MemberAuth"로 지정
- ② 요청을 처리할 서블릿 클래스는 servlet 패키지의 MemberAuth 클래스로 지정
- ③ 서블릿에서 사용할 초기화 매개변수를 지정
 - 관리자 아이디가 "nakja"임을 전달한 것입니다. 앞에서 선언한 <context-param>과 비슷한 설정값이지만 해당 서블릿에서만 쓸 수 있는 것이 차이점
- ④ 요청명은 "/13Servlet/MemberAuth.mvc"

13.6 MVC 패턴을 적용한 회원인증 구현(4)

- [예제 13-15] 회원인증 서블릿
 - ① JDBC 프로그래밍을 위해 6장에서 작성한 MemberDAO 타입의 DAO 객체를 멤버 변수로 선언
 - ② 서블릿을 초기화해줄 init() 메서드를 정의
init() 메서드에서 DB 연결을 위한 DAO 객체를 생성
 - ③ application 내장 객체를 가져오기
 - ④ web.xml에 등록한 컨텍스트 초기화 매개변수 중 DB 연결 정보들을 읽어옴
 - ⑤ 이 정보를 인수로 건네 MemberDAO 객체를 생성
 - ⑥ 클라이언트의 요청을 처리할 service() 메서드를 정의
 - ⑦ [예제 13-14]에서 추가한 서블릿 초기화 매개변수를 얻어옴 - 관리자 아이디로 설정한 값
 - ⑧ JSP에서 매개변수로 전달한 id와 pass를 받음
 - ⑨ member 테이블에서 일치하는 회원(레코드)을 탐색
 - ⑩ 찾은 회원 정보를 기준으로 일치하는 정보가 없을 때,
 - ⑪ 관리자 아이디와 일치할 때,
 - ⑫ 회원 정보와 일치할 때를 구분하여 반환할 메시지를 request 영역에 저장
 - ⑬ MemberAuth.jsp로 포워드
 - ⑭ 서블릿 객체 소멸 시 DAO 객체의 close() 메서드를 호출하여 JDBC에서 사용하던 객체를 메모리에서 소멸시킴

13.6 MVC 패턴을 적용한 회원인증 구현(5)

- MemberAuth.jsp를 실행해서 회원인증 링크들을 클릭하며 동작 확인



학습 마무리

핵심 요약

- 서블릿을 사용하면 MVC 패턴을 적용한 모델2 방식으로 웹 애플리케이션 개발 가능
- 요청명(요청 URL)과 이를 처리할 파일(서블릿)이 분리되어 있어서 둘의 매핑이 필요 (JSP는 요청 URL이 곧 담당 JSP 파일)
- 요청명과의 매핑은 web.xml을 이용하는 방식과 @WebServlet 애너테이션을 이용하는 방식을 제공
- 서블릿은 HttpServlet 클래스를 상속받은 후 요청을 처리할 doGet() 혹은 doPost() 메서드를 오버라이딩하여 제작
- 와일드카드(*)를 사용하여 여러 가지 요청을 하나의 서블릿에서 처리하도록 매핑 가능
- 수명주기 메서드에서 확인했듯이 두 번째 요청부터는 첫 번째 요청 때 만들어둔 객체를 재사용하므로 처리 속도가 빨라짐